

Data Structures — Problems

Summary Table

Data Structure	Description	Key Operations	Types / Variations	Use Cases
Array	Contiguous memory locations storing elements	Access $O(1)$, Search $O(n)$, Insert/Delete $O(n)$	Static, Dynamic	Collections, matrix operations
Hash Table	Key-value pairs with fast retrieval via hashing	Lookup $O(1)$, dynamic sizing, no duplicate keys	—	Databases, caching
Linked List	Nodes with data + pointer to next node	Access $O(n)$, Insert/Delete $O(1)$ at head/tail	Singly, Doubly, Circular	Stacks, queues, memory management
Tree	Hierarchical parent-child structure; one path between any two nodes	Insert/Delete/Search $O(\log n)$	Binary, B-tree, etc.	File systems, database indexing
Queue	FIFO (First-In-First-Out)	Enqueue $O(1)$, Dequeue $O(1)$	Circular, Priority, Deque	Task scheduling, buffering, BFS
Stack	LIFO (Last-In-First-Out)	Push $O(1)$, Pop $O(1)$, Peek $O(1)$	Array or linked list impl.	Function calls, undo/redo, parsing
Graph	Vertices + edges; cycles allowed	Traversal $O(V + E)$	Directed, Undirected, Weighted	Social networks, routing, dependencies

#	Problem	Description	Input	Output
1	Find Largest Number in Array	Given an array of numbers, find and return the largest number.	[1, 5, 3, 9]	9
2	Binary Search in Sorted Array	Given a sorted array of unique integers, find the index of a target integer using binary search $O(\log n)$.	[1, 3, 5, 7, 9], 5	2
3	Sort Array in Ascending Order	Sort an array of integers in ascending order $O(n \log n)$ time, minimal space.	[5, 2, 3, 1]	[1, 2, 3, 5]
4	Two Sum	Find two indices whose values add up to a target value.	[2, 7, 11, 15], 9	[0, 1]
5	Longest Substring Without Repeating Characters	Find the length of the longest substring with all unique characters.	abcabcbb	3
6	Merge Intervals	Merge overlapping intervals into non-overlapping ones.	[[1,3], [2,6], [8,10]]	[[1,6], [8,10]]

#	Problem	Description	Input	Output
7	Group Anagrams	Group words that are anagrams of each other.	<code>["eat", "tea", "tan", "ate", "nat", "bat"]</code>	<code>[["bat"],["nat","tan"],["ate","eat","tea"]]</code>
8	Flatten Nested Arrays	Flatten nested arrays using built-in methods or recursion.	<code>[1, [2, [3, [4, 5]]]]</code>	<code>[1, 2, 3, 4, 5]</code>

1. Find Largest Number in Array

```
function findLargestNumber(arr) {
  let largest = arr[0];
  for (let i = 1; i < arr.length; i++) {
    if (arr[i] > largest) largest = arr[i];
  }
  return largest;
}
```

2. Binary Search in Sorted Array

```
function binarySearch(nums, target) {
  let left = 0, right = nums.length - 1;
  while (left <= right) {
    const mid = Math.floor((left + right) / 2);
    if (nums[mid] === target) return mid;
    else if (nums[mid] < target) left = mid + 1;
    else right = mid - 1;
  }
  return -1;
}
```

3. Sort Array in Ascending Order

```
function sortArray(nums) {
  quickSort(nums, 0, nums.length - 1);
  return nums;
}

function quickSort(nums, left, right) {
  if (left < right) {
    const pivotIndex = partition(nums, left, right);
    quickSort(nums, left, pivotIndex - 1);
    quickSort(nums, pivotIndex + 1, right);
  }
}

function partition(nums, left, right) {
  const pivot = nums[right];
  let i = left - 1;
  for (let j = left; j < right; j++) {
    if (nums[j] <= pivot) {
      i++;
      [nums[i], nums[j]] = [nums[j], nums[i]];
    }
  }
  [nums[i + 1], nums[right]] = [nums[right], nums[i + 1]];
  return i + 1;
}
```

4. Two Sum

Given an array of integers `nums` and an integer `target`, return the indices of the two numbers that add up to `target`.

Why the pointer approach can break

A naive pointer approach often fails because the right pointer moves forward without rechecking earlier elements. That means some pairs are never tested, and the code can go out of bounds.

Approach 1: Brute Force (Nested Loops)

```
function twoSumBruteForce(nums: number[], target: number): number[] {
  for (let i = 0; i < nums.length; i++) {
    for (let j = i + 1; j < nums.length; j++) {
      if (nums[i] + nums[j] === target) {
        return [i, j];
      }
    }
  }
  return [];
}
```

- Time Complexity: $O(n^2)$

- Space Complexity: $O(1)$

Approach 2: Hash Map (Most Common Interview Solution)

```
function twoSumMap(nums: number[], target: number): number[] {
  const map = new Map<number, number>();

  for (let i = 0; i < nums.length; i++) {
    const complement = target - nums[i];

    if (map.has(complement)) {
      return [map.get(complement)!, i];
    }

    map.set(nums[i], i);
  }

  return [];
}
```

- Time Complexity: $O(n)$
- Space Complexity: $O(n)$

Approach 3: Sort + Two Pointers

```
function twoSumSorted(nums: number[], target: number): number[] {
  const indexed = nums.map((value, index) => ({ value, index }));
  indexed.sort((a, b) => a.value - b.value);

  let left = 0;
  let right = indexed.length - 1;

  while (left < right) {
    const sum = indexed[left].value + indexed[right].value;

    if (sum === target) {
      return [indexed[left].index, indexed[right].index];
    } else if (sum < target) {
      left++;
    } else {
      right--;
    }
  }

  return [];
}
```

- Time Complexity: $O(n \log n)$
- Space Complexity: $O(n)$

5. Longest Substring Without Repeating Characters

Given a string `s`, find the length of the longest substring without repeating characters.

Approach 1: Sliding Window with a Set

```
function lengthOfLongestSubstringSet(s: string): number {
  let left = 0;
  let maxLength = 0;
  const seen = new Set<string>();

  for (let right = 0; right < s.length; right++) {
    while (seen.has(s[right])) {
      seen.delete(s[left]);
      left++;
    }

    seen.add(s[right]);
    maxLength = Math.max(maxLength, right - left + 1);
  }

  return maxLength;
}
```

- Time Complexity: $O(n)$
- Space Complexity: $O(k)$

Approach 2: Sliding Window with a Map

```
function lengthOfLongestSubstringMap(s: string): number {
  const lastSeen = new Map<string, number>();
  let left = 0;
  let maxLength = 0;

  for (let right = 0; right < s.length; right++) {
    const char = s[right];

    if (lastSeen.has(char)) {
      left = Math.max(left, lastSeen.get(char)! + 1);
    }

    lastSeen.set(char, right);
    maxLength = Math.max(maxLength, right - left + 1);
  }

  return maxLength;
}
```

- Time Complexity: $O(n)$
- Space Complexity: $O(k)$

6. Merge Intervals

Given an array of intervals `[[start, end]]`, merge all overlapping intervals.

Approach 1: Sort + Greedy

```
function mergeIntervals(intervals: number[][]): number[][] {
  if (intervals.length <= 1) return intervals;

  intervals.sort((a, b) => a[0] - b[0]);

  const merged: number[][] = [intervals[0]];

  for (let i = 1; i < intervals.length; i++) {
    const current = intervals[i];
    const last = merged[merged.length - 1];

    if (current[0] <= last[1]) {
      last[1] = Math.max(last[1], current[1]);
    } else {
      merged.push(current);
    }
  }

  return merged;
}
```

- Time Complexity: $O(n \log n)$
- Space Complexity: $O(n)$

7. Group Anagrams

Given an array of strings, group all anagrams together.

Approach 1: Sort Each String to Create a Key

```
function groupAnagramsSorted(strs: string[]): string[][] {
  const map = new Map<string, string[]>();

  for (const str of strs) {
    const key = str.split('').sort().join('');

    if (!map.has(key)) {
      map.set(key, []);
    }

    map.get(key)!.push(str);
  }

  return Array.from(map.values());
}
```

- Time Complexity: $O(n \cdot k \log k)$
- Space Complexity: $O(n \cdot k)$

Approach 2: Character Frequency Key

```
function groupAnagramsFrequency(strs: string[]): string[][] {
  const map = new Map<string, string[]>();

  for (const str of strs) {
    const counts = new Array(26).fill(0);

    for (const ch of str) {
      counts[ch.charCodeAt(0) - 'a'.charCodeAt(0)]++;
    }

    const key = counts.join('#');

    if (!map.has(key)) {
      map.set(key, []);
    }

    map.get(key)!.push(str);
  }

  return Array.from(map.values());
}
```

- Time Complexity: $O(n \cdot k)$
- Space Complexity: $O(n \cdot k)$

8. Flatten Nested Arrays

Given a nested array, flatten it into a single-level array.

Approach 1: Built-in `.flat()`

```
function flattenWithFlat(arr: any[]): any[] {
  return arr.flat(Infinity);
}

const nested = [1, [2, [3, [4, 5]]]];
console.log(flattenWithFlat(nested));
// Output: [1, 2, 3, 4, 5]
```

- Time Complexity: $O(n)$
- Space Complexity: $O(n)$

Approach 2: Recursive Flattening

```
function flattenRecursive<T>(arr: any[]): T[] {  
  let result: T[] = [];  
  
  for (const item of arr) {  
    if (Array.isArray(item)) {  
      result.push(...flattenRecursive<T>(item));  
    } else {  
      result.push(item as T);  
    }  
  }  
  
  return result;  
}
```

- Time Complexity: $O(n)$
- Space Complexity: $O(n)$